FAKULTA INFORMATIKY A STATISTIKY

Ing. Karel Šafr, Ph.D.

Python před pygame

Opakování Pythonu před tvorbou her

Vysoká škola ekonomická v Praze

VŠE / FAKULTA INFORMATIKY A STATISTIKY

Co společně projdeme

U ukázek nejprve odhadni výsledek a pak změň jeden vstup.

Část	Co budeme dělat
Hodnoty a výrazy	Čísla, text a použití modulů
Kontejnery	Souřadnice, seznamy, slovníky a množiny
Řízení programu	Podmínky a průchody cyklem
Funkce	Argumenty a návratová hodnota
Objekty	Stav, metody a spolupráce objektů
Společná úloha	Zpracování příkazů k pohybu

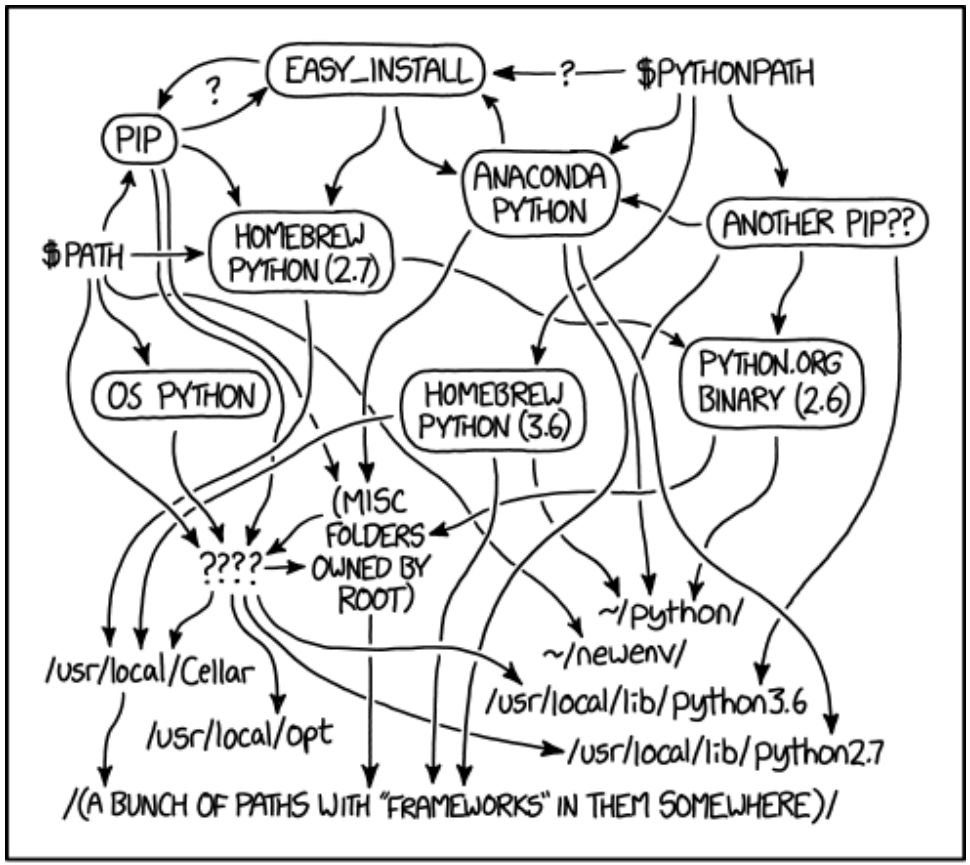
VŠE / FAKULTA INFORMATIKY A STATISTIKY

Buňka, výstup a paměť

Kód vykoná kernel; výstup je záznam spuštění.
Po úpravě kódu buňku znovu spusťte.

```
jmeno = "Alex"
print("Připraveno na Python.")
print("Hráč:", jmeno)
```

Prostředí Pythonu



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

Python může být v počítači nainstalovaný víckrát.
Kernel používá jednu konkrétní instalaci.
Při potížích ověřte, ve kterém prostředí běžíte.

Jak číst chybu

Záměrné chyby: zivotty není definováno, mana chybí.
Seznam má tři prvky; úryvky slouží jen k rozboru.

Chyba	Chybný text	Co ověřit
NameError	print(zivotty)	Jméno a přiřazení
TypeError	"3" + 1	Text oproti číslu
IndexError	seznam[3]	Délku seznamu
KeyError	hrac["mana"]	Existenci klíče
SyntaxError	if zivoty > 0	Dvojtečku

Jména a hodnoty

Jméno proměnné odkazuje na objekt.
Typ popisuje, s jakou hodnotou pracujeme.

```
jmeno = "Alex"
x = 120
y = 80
zivoty = 3
rychlost = 2.5
pauza = False
print(type(x))
print(type(jmeno))
```

Přiřazení a změna hodnoty

Nejprve se vyhodnotí pravá strana, potom se přiřadí.
Pohyb používá + a -; více stejných kroků násobení *.

```
x = 10
x = x + 5
print("Po prvním posunu:", x)
x += 5
print("Po druhém posunu:", x)
```

Podíl, zbytek a mocnina

// dává podíl zaokrouhlený dolů; % zbytek po dělení.

Zápis	Význam	Výsledek
10 / 3	Desetinný podíl	3.333…
10 // 3	Podíl zaokrouhlený dolů	3
10 % 3	Zbytek po dělení	1
3 ** 2	Mocnina: 3 krát 3	9
-10 // 3	Dolů, tedy směrem k $-\infty$	-4
-10 % 3	-10 = (-4) * 3 + 2	2

Úloha 1: posun hráče

Nahraď pouze 0 na řádku x += 0 výpočtem.
Použijte rychlost a pocet_kroku; očekávané x je 135.
Potom ověřte nula kroků a jeden krok.

```
x = 120
rychlost = 5
pocet_kroku = 3
x += 0  # TODO: celkový posun
print("Konečné x:", x)
```

Číslo a text

Uvozovky vytvářejí text; int převádí platný text na číslo.

```
cislo = 3
text = "3"
print("Výpočet:", cislo + 1)
print("Spojení textů:", text + "1")
print("Převod a výpočet:", int(text) + 1)
print("Typ 3.0:", type(3.0))
```

Mikroověření 1

Odhadněte hodnotu i typ každého výsledku.
Chybný text k rozboru: "3" + 2.
Který výsledek je text a proč?

```
print(3 + 2)
print("3" + "2")
print(int("3") + 2)
```

Ladění 1: životy jako text

Chybný výraz k rozboru: "3" - 1.
Do TODO doplňte převod a odečtení; očekáváme 2.

```
zivoty = "3"
# TODO: převod a odebrání jednoho života
print("Životy:", zivoty)
```

Závorky a tečka

V příkladech má jmeno hodnotu "Alex".

Zápis	Jak jej čtu
print(jmeno)	Vypiš hodnotu proměnné.
print("jmeno")	Vypiš doslovný text.
jmeno[0]	Vyber první znak.
jmeno[:3]	Vyber indexy 0, 1, 2.
jmeno.upper()	Zavolej metodu tohoto textu.

Metody a výřezy řetězců

Metody vracejí výsledek; původní text se nemění.

```
text = "hráč Alex má tři životy"
print(text.upper())
print(text.replace("tři", "dva"))
print(text.split())
print(text.find("Alex"))
print(text[::2])
print("105".isnumeric(), "10.5".isnumeric())
```

Importy

import zpřístupní modul nebo vybrané jméno či skript (což je ale modul)

```
import math
from random import randrange

print(math.hypot(3, 4))
nepritel_x = randrange(0, 800)
print(nepritel_x)
print(0 <= nepritel_x < 800)
```

Varianty importu

Alias mění dostupné jméno, nikoli importovanou funkci.
from math import * zpřístupní veřejná jména z math.

```
import math
import math as matematika
from math import hypot
from math import hypot as vzdalenost

print(math.hypot(3, 4))
print(matematika.hypot(3, 4))
print(hypot(3, 4))
print(vzdalenost(3, 4))
```

Klíčová slova a názvy

Klíčová slova určuje používaná verze Pythonu.
Seznam poskytují keyword.kwlist a keyword.softkwlist.
Názvy list, str nebo input nepřepisuj proměnnými.

```
import keyword
print(keyword.iskeyword("while"))
print(keyword.iskeyword("zivoty"))
```

Číselné soustavy a komplexní čísla

Předpona změní zápis čísla; výsledný typ je stále int.
Komplexní číslo zapisuje imaginární část pomocí j.

```
print(0b11111111, 0o377, 0xFF)
print(bin(255), oct(255), hex(255))

cislo = complex("1+2j")
print(cislo.real, cislo.imag)
```

Desetinná přesnost a zlomky

float ukládá mnoho desetinných hodnot přibližně.
Decimal zde tvoříme z textu; Fraction uchová zlomek.

```
from decimal import Decimal
from fractions import Fraction

print(0.1 + 0.2)
print(Decimal("0.1") + Decimal("0.2"))
print(Fraction(1, 3) + Fraction(1, 6))
```

Náhodné hodnoty a hranice

Horní hranici čtěte podle konkrétní funkce.

Volání	Možný výsledek
randrange(0, 100)	Celé číslo 0 až 99
randint(0, 100)	Celé číslo 0 až 100
random()	0.0 včetně až 1.0 bez jedničky
Random(7)	Vlastní generátor se zvoleným seedem

Přehled kontejnerů

Iterovatelný objekt dovoluje postupně procházet prvky.

Typ	Měnitelný	Přístup a pořadí
list	ano	Index; zachovává pořadí
tuple	ne	Index; zachovává pořadí
dict	ano	Klíč; pořadí vložení
set	ano	Členství; bez záruky pořadí

Souřadnice a barvy

N-tice tuple uchovává uspořádané hodnoty.
Její položky nepřepisujeme; tvoříme novou n-tici.

```
pozice = (120, 80)
barva = (255, 128, 0)
x, y = pozice
print(x, y)
print("Zelená složka:", barva[1])
nova_pozice = (x + 5, y)
print(pozice, nova_pozice)
```

Rozbalení a prohození hodnot

Nejprve se vyhodnotí celá pravá strana přiřazení.
Dvě původní hodnoty se potom přiřadí v novém pořadí.

```
x, y = 10, 20
x, y = y, x
print(x, y)
```

Index není hodnota

Seznam: ["sliz", "duch", "robot"].

Co sledujeme	První prvek	Druhý prvek	Třetí prvek
Index	0	1	2
Hodnota	sliz	duch	robot
Záporný index	-3	-2	-1

Čtení a změna seznamu

append přidá prvek; remove odebere první danou hodnotu.

```
nepratele = ["sliz", "duch", "robot"]
print(nepratele[0])
nepratele.append("kostlivec")
print(nepratele)
nepratele.remove("sliz")
print(nepratele)
```

Úloha 2: seznam nepřátel

Do TODO přidejte robota a odeber sliz podle hodnoty.
Očekávaný seznam je ["duch", "robot"].
Ověřte také ["sliz"] a ["sliz", "sliz", "duch"].

```
nepratele = ["sliz", "duch"]
# TODO: přidej robota a odeber sliz
print(nepratele)
```

Slovník hráče

Slovník dict spojuje klíč s hodnotou.
Klíč "zivoty" není číselný index.

```
hrac = {"x": 10, "y": 20, "zivoty": 3}
hrac["x"] += 5
hrac["zivoty"] -= 1
hrac["skore"] = 10
print(hrac)
print("Životy:", hrac["zivoty"])
```

Mikroověření 2

Odhadněte výsledek výběru indexem a klíčem.
Proč hrac[1] neoznačuje druhou hodnotu?

```
nepratele = ["sliz", "duch"]
hrac = {"x": 10, "zivoty": 3}
print(nepratele[1])
print(hrac["zivoty"])
```

Množina a jedinečné hodnoty

Množina set uchovává jedinečné prvky bez garantovaného pořadí.
Prázdnou množinu vytvoří set(), prázdný slovník {}.
Prvky musejí být hashovatelné.

```
predmety = {"klic", "mince", "mince"}
print(len(predmety))
print("klic" in predmety)
print(predmety & {"klic", "mapa"})
```

Co může být prvkem množiny

Hashovatelnost umožňuje spolehlivě hledat prvek.
Stejná podmínka platí také pro klíč slovníku.

Příklad	Lze použít?	Proč
10 nebo "klic"	ano	Čísla a text jsou hashovatelné.
(10, 20)	ano	Hashovatelné jsou i obě položky.
[10, 20]	ne	Seznam není hashovatelný.
{"x": 10}	ne	Slovník není hashovatelný.
([10, 20],)	ne	Obsahuje nehashovatelný seznam.

Operace s množinami

Operace vytvářejí množinu odpovídajících prvků.

Zápis	Význam
a | b	Sjednocení: prvky alespoň jedné množiny
a & b	Průnik: společné prvky
a - b	Rozdíl: prvky a, které nejsou v b
a ^ b	Symetrický rozdíl: prvky právě jedné

Druhé jméno a kopie

Přiřazení seznam nekopíruje.
is porovnává totožnost, == porovnává hodnoty.

```
nepratele = ["sliz", "duch"]
druhe_jmeno = nepratele
kopie = nepratele.copy()
druhe_jmeno.append("robot")
print(nepratele)
print(kopie)
print(druhe_jmeno is nepratele)
```

Měnitelné a neměnitelné objekty

Měnitelnost popisuje objekt, nikoli jméno proměnné.

Měnitelné	Neměnitelné
list, dict, set	int, float, bool, complex
bytearray	str, tuple, frozenset
Lze změnit obsah na místě.	Změna vyžaduje jinou hodnotu.

Neměnnost a vnořené objekty

N-tice nemůže vyměnit svoje položky.
Obsah měnitelného objektu uvnitř n-tice se změnit může.
Mělká kopie stále sdílí vnořené objekty.

```
souradnice = [10, 20]
balik = (souradnice,)
souradnice[0] = 99
print(balik)
print(balik[0] is souradnice)
```

Přestávka

Po návratu navážeme podmínkami a cykly.

Podmínka jako otázka

Porovnání vrací True nebo False.
Co se změní při zivoty = 0?

```
zivoty = 2
print(zivoty > 0)
print(zivoty <= 0)
print(zivoty == 2)
```

Jednoduché if a odsazení

Odsazené řádky patří do podmínky.
Neodsazený výpis se provede vždy.
Před spuštěním porovnej životy 0 a 1.

```
x = 10
zivoty = 0
if zivoty > 0:
    x += 5
    print("Pohyb proběhl.")
print("Tento výpis je mimo if. x =", x)
```

Více možností

if, elif a else vyberou nejvýše jednu větev.

```
zivoty = 2
pauza = False
if zivoty <= 0:
    print("Konec hry")
elif pauza:
    print("Pauza")
else:
    print("Hráč může pokračovat")
```

Spojování podmínek

and vyžaduje obě podmínky; or alespoň jednu.
not obrátí pravdivost.

```
zivoty = 2
pauza = False
muze_se_hybat = zivoty > 0 and not pauza
print("Pohyb povolen:", muze_se_hybat)
print("Je třeba čekat?", zivoty <= 0 or pauza)
```

Úloha 3: povolený pohyb

Nahraďte False podmínkou pro povolený pohyb.
Dva životy bez pauzy mají dát x = 15.
Ověřte nula životů, pauzu a záporné životy.

```
x = 10
rychlost = 5
zivoty = 2
pauza = False
if False:  # TODO: povolený pohyb
    x += rychlost
print("x:", x)
```

Přiřazovací výraz :=

:= přiřadí hodnotu a zároveň ji poskytne uvnitř výrazu.
Závorky zde oddělí přiřazení délky od porovnání.

```
nepratele = ["sliz", "duch"]
if (pocet := len(nepratele)) > 0:
    print("Počet nepřátel:", pocet)
```

For prochází hodnoty

Proměnná nepritel postupně dostává jednotlivé prvky.

```
nepratele = ["sliz", "duch", "robot"]
for nepritel in nepratele:
    print("Nepřítel:", nepritel)
```

Index spolu s hodnotou

enumerate poskytne dvojici index a hodnota.

```
nepratele = ["sliz", "duch", "robot"]
for index, nepritel in enumerate(nepratele):
    print(index, nepritel)

hrac = {"x": 10, "y": 20, "zivoty": 3}
for klic, hodnota in hrac.items():
    print(klic, hodnota)
```

Range a počet opakování

Koncová hodnota stop se nezahrnuje.

```
x = 10
rychlost = 5
for krok in range(3):
    x += rychlost
    print("Krok:", krok, "| x:", x)
print(list(range(3)))
print(list(range(1, 4)))
print(list(range(0, 6, 2)))
```

Pohyb po jednotlivých krocích

Začínáme x = 10; rychlost = 5; opakujeme range(3).

krok	x před změnou	x po změně
0	10	15
1	15	20
2	20	25

Mikroověření 3

Změňte rychlost na 2 a ručně sleduj tři průchody.
Jaké bude konečné x při range(0)?

```
x = 10
rychlost = 2
for krok in range(3):
    x += rychlost
    print("Krok:", krok, "| x:", x)
```

Ladění 2: poslední index

Chybné úryvky jsou pouze text k rozboru.
Ve startu ladění 2 doplň enumerate; očekávej 0, 1, 2.

Chybný text	Problém u tří prvků
range(1, 4)	Vynechá 0 a zahrne 3.
nepratele[3]	Index 3 neexistuje.

While a čtyři snímky

Podmínku kontrolujeme před každým provedením těla.

```
x = 0
rychlost = 5
pocet_snimku = 0
maximum_snimku = 4
while pocet_snimku < maximum_snimku:
    x += rychlost
    pocet_snimku += 1
    print("Snímek:", pocet_snimku, "| x:", x)
print("Smyčka skončila.")
```

Poslední kontrola while

Po posledním průchodu zkoušíme podmínku ještě jednou.

Počet před	Je menší než 4?	x po rozhodnutí
0	True	5
1	True	10
2	True	15
3	True	20
4	False	20

Continue a break

continue přeskočí zbytek průchodu; break ukončí cyklus.

```
x = 0
prikazy = ["vpravo", "cekej", "vpravo",
           "konec", "vpravo"]
for prikaz in prikazy:
    if prikaz == "cekej":
        continue
    if prikaz == "konec":
        break
    x += 5
    print("Posun:", x)
```

Úloha 4: tři kroky doleva

Do TODO doplňte celý cyklus for s range.
Tři původní kroky mají dát x = 18.
Ověřte nula kroků a jeden krok.

```
x = 30
rychlost = 4
# TODO: tři kroky doleva pomocí for
print("Konečné x:", x)
```

Od cyklu k vytvoření seznamu

append přidá každý vypočítaný výsledek do seznamu.
Comprehension zapíše tentýž postup na jednom řádku.

```
pozice_x = [0, 10, 20, 30]
posunute_cyklem = []
for x in pozice_x:
    posunute_cyklem.append(x + 5)
posunute_kratce = [x + 5 for x in pozice_x]
print(posunute_cyklem)
print(posunute_kratce)
```

Comprehension s podmínkou

Comprehension vytvoří celou kolekci.
Čtěte jako: vezmi z pro každý prvek, který splní z > 0.

```
zivoty = [3, 0, 2, 0, 1]
zivi = [z for z in zivoty if z > 0]
print(zivi)
```

Slovník a množina pomocí comprehension

Stejný princip může vytvořit i slovník nebo množinu.

```
pozice_x = [0, 10, 20, 30]
index_na_x = {i: x for i, x in enumerate(pozice_x)}
ruzne_zbytky = {x % 20 for x in pozice_x}
print(index_na_x)
print(sorted(ruzne_zbytky))
```

Generátor poskytuje hodnoty postupně

Generátorový výraz poskytuje hodnoty postupně.
První tuple generátor vyčerpá; podruhé už je prázdný.

```
zivoty = [3, 0, 2, 0, 1]
generator = (z for z in zivoty if z > 0)
print(tuple(generator))
print(tuple(generator))
```

Jeden cyklus uvnitř druhého

Vnější cyklus vybírá řádek, vnitřní jeho položky.
Každý nový řádek začíná novým prázdným seznamem.

```
mrizka = []
for y in range(2):
    radek = []
    for x in range(3):
        radek.append(x + y)
    mrizka.append(radek)
print(mrizka)
```

Stejná mřížka kratším zápisem

Vnitřní comprehension vytvoří řádek; vnější řádky.
Podrobný rozklad obou variant je v notebooku.

```
mrizka = [
     [x + y for x in range(3)]
     for y in range(2)
]
print(mrizka)
```

Párování pomocí zip

zip páruje odpovídající položky vstupů.
Běžně končí s nejkratším vstupem.

```
jmena = ["Alex", "Bára"]
barvy = ["oranžová", "modrá"]
for jmeno, barva in zip(jmena, barvy):
    print(jmeno, barva)

print(list(zip([1, 2, 3], ["A", "B"])))
```

Else po cyklu

else u cyklu proběhne při dokončení bez break.
Patří k for, nikoli k vnitřnímu if.

```
nepratele = ["sliz", "duch"]
hledany = "robot"
for nepritel in nepratele:
    if nepritel == hledany:
        print("Nalezen:", hledany)
        break
else:
    print("Nenalezen:", hledany)
```

Definice a volání funkce

def uloží předpis; tělo provede až volání.
return předá výsledek na místo volání.

```
def posun(x, rychlost):
    nove_x = x + rychlost
    return nove_x

puvodni_x = 10
vysledek = posun(puvodni_x, 5)
print("Původní číslo:", puvodni_x)
print("Vrácený výsledek:", vysledek)
```

Cesta argumentu a výsledku

Sledujte vysledek = posun(puvodni_x, 5).

Krok	Co se děje
Argumenty	Přečteme 10 a 5.
Parametry	x = 10, rychlost = 5.
Výpočet	nove_x dostane 15.
return	Hodnota 15 se vrací volajícímu.
Přiřazení	vysledek dostane 15.

Print oproti return

Výpis na obrazovku není návratová hodnota.
Funkce bez return vrátí None.

```
def vypis_pozici(x):
    print("Pozice:", x)
def posun(x, rychlost):
    return x + rychlost

vysledek_vypisu = vypis_pozici(10)
nove_x = posun(10, 5)
print(vysledek_vypisu)
print(nove_x)
```

Výchozí argumenty a dvojice

Parametr může mít výchozí hodnotu.
Vrácenou n-tici můžeme rozbalit.

```
def posun_pozici(x, y, dx=5, dy=0):
    return x + dx, y + dy

print(posun_pozici(10, 20))
nove_x, nove_y = posun_pozici(10, 20, dx=0, dy=-3)
print(nove_x, nove_y)
```

Omezení hodnoty pomocí max a min

max vybere větší hodnotu; min vybere menší.

Zápis	Výsledek	Použití
max(0, -4)	0	Dolní mez nula
max(0, 7)	7	Kladná hodnota zůstane
min(3, 8)	3	Horní mez tři
min(3, 1)	1	Nižší hodnota zůstane

Úloha 5: ztráta životů

Doplňte tělo funkce: odečti zásah a vrať nejméně 0.
Použijte if nebo max; uvnitř funkce nic nevypisujte.
Očekáváme 2 a 0; vstupy jsou nezáporné celé počty.

```
def uber_zivoty(zivoty, zasah=1):
    return zivoty  # TODO: doplň tělo funkce

print(uber_zivoty(3))
print(uber_zivoty(2, zasah=5))
```

Vestavěné funkce při zkoumání kódu

Výpis, návratová hodnota a načtení vstupu jsou různé věci.

Funkce	Příklad	Co poskytuje
type	type(3)	Typ objektu
repr	repr("Alex\nBára")	Textovou reprezentaci
id	id(objekt)	Identitu po dobu života
divmod	divmod(10, 3)	Dvojici (3, 1)
input	input("Počet životů: ")	Načte řádek jako str
help	help(str.upper)	Zobrazí nápovědu

VŠE / FAKULTA INFORMATIKY A STATISTIKY

Sesbírání argumentů do kontejneru (kw)args

*hodnoty sesbírá poziční argumenty do n-tice.
**vlastnosti sesbírá pojmenované argumenty do slovníku.

```
def seber_hodnoty(*hodnoty):
    return hodnoty
def seber_vlastnosti(**vlastnosti):
    return vlastnosti

print(seber_hodnoty(10, 20, 5))
print(seber_vlastnosti(jmeno="Alex", zivoty=3))
```

Vložení hodnot do textu

f před uvozovkou umožní vložit hodnoty pomocí {…}.
join spojí textové části; sum sečte číselné hodnoty.

```
jmeno = "Alex"
zivoty = 3
print(f"{jmeno}: {zivoty}")
casti = ["jmeno=Alex", "zivoty=3"]
print(", ".join(casti))
print(sum((10, 20, 5)))
```

Proměnlivý počet argumentů

*hodnoty vytvoří n-tici; **vlastnosti slovník.

```
def secti_skore(*hodnoty):
    return sum(hodnoty)
def popis_hrace(**vlastnosti):
    casti = (f"{k}={v}" for k, v in vlastnosti.items())
    return ", ".join(casti)

print(secti_skore(10, 20, 5))
print(popis_hrace(jmeno="Alex", zivoty=3))
```

Typové anotace a návrat dvojice

Anotace pomáhají čtenáři; samy typy za běhu nevynucují.
a: int označuje typ parametru; -> uvádí typ výsledku.
Dvojici celých čísel označuje tuple[int, int].

```
def deleni_a_zbytek(a: int, b: int) -> tuple[int, int]:
    return a // b, a % b

podil, zbytek = deleni_a_zbytek(10, 3)
print(podil, zbytek)
print(divmod(10, 3))
```

Přestávka

Po návratu spojíme stav a chování do třídy.

Od slovníku k instanci

Třída je předpis; instance je konkrétní objekt.
Stejný stav hráče můžeme číst přes atributy instance.

Účel	Slovník	Instance
Souřadnice x	hrac["x"]	alex.x
Životy	hrac["zivoty"]	alex.zivoty
Posun x	hrac["x"] += 5	alex.x += 5
Obě souřadnice	vlastní funkce	alex.posun(5, 0)

Definice třídy Hrac

__init__ nastaví stav; posun jej později mění.
self je konvenční jméno parametru pro instanci.

```
class Hrac:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.zivoty = 3

    def posun(self, dx, dy):
        self.x += dx
        self.y += dy
```



Jak se předává self

Hrac(10, 20) vytvoří instanci; __init__ nastaví stav.
Při volání metody Python předá instanci automaticky.

Volání	Parametry při provedení
alex = Hrac(10, 20)	self = nová instance, x = 10, y = 20
alex.posun(5, 0)	self = alex, dx = 5, dy = 0
bara.posun(5, 0)	self = bara, dx = 5, dy = 0

Vytvoření jedné instance

Použijte předchozí definici třídy Hrac.
Atribut čteme; metodu voláme závorkami.

```
alex = Hrac(10, 20)
print(alex.x, alex.y, alex.zivoty)
alex.posun(5, 0)
print(alex.x, alex.y)
```

Dvě instance, dva stavy

Použijte stejnou definici Hrac.
Alexův pohyb a zásah mění pouze Alexe.

```
alex = Hrac(10, 20)
bara = Hrac(100, 20)
alex.posun(5, 0)
alex.zivoty -= 1
print("Alex:", alex.x, alex.y, alex.zivoty)
print("Bára:", bara.x, bara.y, bara.zivoty)
```

Úloha 6: léčení hráče

V notebooku nahraďte pass v HracSLecenim.vylec.
Přidejte jeden život, nejvýše do 3; použijte if nebo min.
pass zatím drží místo pro kód a nic neprovede.

Zkouška	Alex před	Alex po	Bára po
První léčení	2	3	1
Další léčení	3	3	1
Léčení od nuly	0	1	1

Atribut může přibýt za běhu

Python spoléhá také na konvence rozhraní objektu.
U běžné instance můžeme přidat další atribut.

Zápis	Jak jej chápat
objekt.jmeno	Běžně veřejné rozhraní
objekt.skore = 0	Může přidat nový atribut instance

Atribut třídy a instance

Třídní atribut poskytuje společnou výchozí hodnotu.
Atribut instance může stejné jméno překrýt.
Atributy nejsou automaticky konstanty.

```
class Nastaveni:
    rychlost = 5
a = Nastaveni()
b = Nastaveni()
a.rychlost = 9
Nastaveni.rychlost = 6
print(a.rychlost, b.rychlost)
```

Sdílený třídní seznam

Mutace změní sdílený objekt; přiřazení jej může překrýt.

```
class UkazkaSdileni:
    seznam = []
a = UkazkaSdileni()
b = UkazkaSdileni()
a.seznam.append("pohyb")
print(b.seznam)
a.seznam = ["vlastní seznam"]
print(a.seznam, b.seznam)
```

Označení metody pomocí @

Řádek s @ upravuje následující definici metody.
@classmethod zajistí automatické předání třídy.
@staticmethod nepředává ani instanci, ani třídu.

```
class Kontrola:
    @staticmethod
    def platna_rychlost(rychlost):
        return rychlost >= 0

print(Kontrola.platna_rychlost(5))
```

Druhy metod

Instanční metoda dostává instanci (obvykle self).
Třídní metoda dostává třídu (obvykle cls).
Statická metoda nemá automatický první argument.

```
class Nastaveni:
    vychozi_zivoty = 3

    @classmethod
    def pocet_zivotu(cls):
        return cls.vychozi_zivoty

print(Nastaveni.pocet_zivotu())
```

Konvence neveřejných jmen

Podtržítko vyjadřuje záměr autora rozhraní.
Name mangling přepíše jméno, aby omezil kolize.

Zápis	Jak jej chápat
objekt._jmeno	Konvence neveřejné součásti
__jmeno ve třídě	Name mangling proti kolizím
Ve třídě Hrac: __body	Interní jméno _Hrac__body

Dědičnost přebírá chování

Použij definici Hrac z předchozí části.
Nepritel dědí __init__ i posun; pass nic neprovede.

```
class Nepritel(Hrac):
    pass

nepritel = Nepritel(100, 20)
nepritel.posun(5, 0)
print(nepritel.x, nepritel.y)
```

Vytvoření instance a nastavení stavu

Při Hrac(10, 20) vznikne nový objekt a nastaví se jeho stav.
Pro běžnou třídu obvykle píšeme jen __init__.

Krok	Metoda	Úloha
1	__new__	Vytvoří a vrátí novou instanci.
2	__init__	Nastaví stav již existující instance.

Speciální metody objektu

Python používá speciální metody pro známé operace.

Operace	Metoda	Význam
Hrac(10, 20)	__init__	Inicializace stavu
str(vektor)	__str__	Textový popis objektu
vektor_a + vektor_b	__add__	Vlastní význam sčítání

Pojmenované argumenty a rozbalení

Samostatná * v definici vyžaduje další argumenty jménem.
**slovnik při volání rozbalí položky na argumenty.

```
def vytvor_stav(*, jmeno, x=0, y=0):
    return {"jmeno": jmeno, "x": x, "y": y}

nastaveni = {"jmeno": "Alex", "x": 10, "y": 20}
print(vytvor_stav(jmeno="Alex", x=10, y=20))
print(vytvor_stav(**nastaveni))
```

Sesbírat a předat argumenty dál

Použijte předchozí definici vytvor_stav.
**kwargs v definici sbírá; při volání rozbaluje.

```
def predej_stav(**kwargs):
    return vytvor_stav(**kwargs)

print(predej_stav(jmeno="Bára", x=100, y=20))
```

Jedna třída se dvěma rodiči

Použijte rodiče Pojmenovany a Pohyblivy z notebooku.
Pořadí rodičů ovlivňuje pořadí vyhledávání metod.

```
class PohyblivyHrac(Pojmenovany, Pohyblivy):
    pass

hrac = PohyblivyHrac(jmeno="Alex", x=10, y=20)
hrac.posun(5)
print(hrac.jmeno, hrac.x, hrac.y)
```

Vícenásobná dědičnost a super

Více rodičů vyžaduje společný postup inicializace.
super() pokračuje podle pořadí vyhledávání metod MRO.

Pořadí v ukázce notebooku	Role
PohyblivyHrac	Spojí vlastnosti obou rodičů
Pojmenovany	Zpracuje jmeno
Pohyblivy	Zpracuje x a y
object	Dokončí inicializaci

Co předává super dalšímu v pořadí

Každá metoda odebere svoje argumenty.
super().__init__(**kwargs) předá zbytek další metodě.

Metoda třídy	Zpracuje	Předá dál
Pojmenovany	jmeno = "Alex"	x = 10, y = 20
Pohyblivy	x = 10, y = 20	Prázdné kwargs
object	Žádné argumenty	Inicializace končí

Skládání propojuje objekty

Použijte definici Hrac z předchozí části.
Hra má hráče a seznam nepřátel; nemusí z nich dědit.

```
class Nepritel(Hrac):
    pass

class Hra:
    def __init__(self):
        self.hrac = Hrac(10, 20)
        self.nepratele = [Nepritel(100, 20)]
```

Stejné volání, jiné chování

Různé objekty mohou podporovat stejné potřebné rozhraní.
Společný kód pak nemusí znát jejich konkrétní třídu.
Výsledky se mohou lišit; rozhoduje kompatibilní chování.

```
class Straz:
    def update(self):
        return "Hlídám"
class Duch:
    def update(self):
        return "Mizím"
for objekt in (Straz(), Duch()):
    print(objekt.update())
```

Úloha 7: zpracování pohybů

Doplňte tělo funkce ve startu úlohy 7 v notebooku.
Rychlost je nezáporná velikost kroku.

```
def zpracuj_pohyby(x, prikazy, rychlost=5):
    # TODO: cyklus, podmínky a ukončení
    return x
```

První fáze: samotný pohyb

Zpracujte nejprve jen tři pohybové příkazy.

Vstup	Hodnota
Začátek x	10
Rychlost	5
Příkazy	vpravo, vpravo, vlevo
Očekávaný výsledek	15

Druhá fáze: konec a čekání

Přidej ukončení a ignorování neznámého povelu.

Příkazy při x = 10	Výsledné x
konec, vpravo	10
cekej	10
vpravo, cekej	15

Trasování vlastního řešení

Doplňte x před a po příkazu; start je 10, rychlost 5.
Řádky za konec se už neprovedou; nepočítejte je.

Příkaz	x před	x po
vpravo	10	?
vpravo	?	?
vlevo	?	?
konec	?	?
vpravo	—	neprovedeno

Kontrola návratové hodnoty

return x patří za cyklus, stále však do funkce.
Každou zkoušku začni na x = 10.

Zkouška	Očekávané x
Prázdný seznam	10
konec jako první	10
Rychlost 0	10
vpravo, vpravo	20
vlevo, vlevo	0

Připraveno na pygame

Umím přečíst stav a předpovědět změnu.

Použiji podmínku, cyklus a návrat funkce.

Rozliším třídu, instanci, atribut a volání metody.